

Smart Inbox Assistant for a Healthcare Company

Build a working application that reads incoming emails and PDF attachments, figures out what each one is about, pulls out the key facts, and hands everything to a human reviewer to check.

Field	Detail
Time to Submit	7 calendar days from receipt
Confidentiality	Internal / candidate use only — do not share externally

1. What This Assignment Is About

Clinevo builds software for pharmaceutical companies that helps them keep track of patient safety. A big part of that job is simple to describe, even though the industry sounds complicated:

Every day, a shared company mailbox receives emails and PDF attachments from doctors, patients, and other sources. Someone has to read every single one, figure out what kind of message it is, and pull out the important details — by hand. It's slow and error-prone.

Your job: build a working prototype that automates this first pass, using AI to read, sort, and summarize incoming messages so a human reviewer only has to check and confirm the AI's work, not start from scratch.

You do not need any pharma background to do this well. Everything you need to know is explained below in plain English. This is fundamentally a document-understanding and classification problem — the kind of system you'd build for insurance claims, support tickets, or legal intake, just applied to healthcare mail.

2. The Four Simple Categories

Every incoming email (with its attachments) needs to be sorted into one or more of these four buckets. A short definition is given for each — no prior knowledge assumed.

Category	Plain-English meaning	Look for...
Safety Report ("ICSR")	A patient had a bad reaction to a drug.	A specific patient, a specific person reporting it, a specific drug, and a bad outcome — all four present, even loosely.
Quality Complaint ("PQC")	Something is physically wrong with the product itself.	Words like broken seal, wrong color, contamination, damaged packaging, counterfeit.
Info Request ("MI")	Someone just has a question about a product.	Questions about dosing, how to take it, interactions — with no bad reaction and no defect.
Not Relevant	Anything else.	Marketing, spam, internal admin chatter.

A message can land in more than one bucket at once (e.g., a bad reaction caused by a defective product) — don't force a single label.

3. What the App Must Do

A. Read the mail

- Accept incoming messages —connect to a real test mailbox

- Pull out the sender, subject, date, and body text of each email.
- Grab every PDF attachment for processing (other file types can just be logged, not processed).
- Save every processed message and its results somewhere queryable (Oracle database — see Section 5).

B. Understand the PDFs

Attached PDFs won't all look the same. Your app should detect which of these four "flavors" a PDF is, and handle each one appropriately:

PDF type	What to do
Normal digital PDF	Extract the text directly, keeping form fields/labels lined up correctly.
Scanned or handwritten	Use OCR / a vision-capable AI model to read it, and show a confidence score since handwriting is uncertain.
Published article	Handle multi-column layouts and pull out the parts that describe an actual patient case, ignoring references and general discussion.
Non-English	Detect the language and translate to English (or extract directly in that language — your choice, just explain it). Keep a link back to the original text.

Within any PDF, also:

- Pull tables (e.g., lab values, dosing schedules) into structured rows/columns — not just flattened text.
- For meaningful images (a photo of a damaged product, a rash, a filled-in checkbox form), write a short text description and flag it for human review. Deep image analysis isn't required — just a good-faith attempt.
- Write a short AI-generated summary (10–15 sentences) of each PDF for a human reviewer, saying whether it looks relevant and why.

C. Sort and score each message

- Classify each message into one or more of the 4 buckets from Section 2, with a confidence score and a one-line reason for each.
- Build a simple review screen (Angular) listing incoming items with the AI's classification, confidence, and summary — and let a reviewer accept or override it.

D. Pull out the key facts

For anything sorted as a Safety Report, extract these details (mark anything not mentioned as "Not stated" rather than guessing):

Group	Examples of fields
Patient	Age, sex, weight/height, relevant history
Reporter	Who reported it, their role, country
Product	Name, dose, route, start/stop dates
Reaction	What happened, when it started, outcome
Severity	Was it serious — death, hospitalization, life-threatening, etc.
Narrative	A short AI-written case summary in plain language

- Every extracted fact must link back to exactly where it came from (which email or which PDF page) — this is required, not optional, for audit purposes.
- For Quality Complaints: pull out product/batch/lot number, what's wrong, and whether a photo was mentioned.
- For Info Requests: pull out the actual question(s) being asked and what product/topic it's about.

E. A few ground rules

Rule	Why
Say "unknown" instead of guessing.	A wrong guess is worse than an honest gap. Show a confidence score for every extracted field.
Log everything.	Every AI decision should be traceable back to the input that produced it, and every reviewer action should be timestamped.
No real patient data — ever.	Use only made-up/synthetic test data. If you use a cloud AI API, briefly note the data-handling trade-off.
Handle a real batch.	Process at least 10–15 sample documents automatically and report how long each one takes.

4. Optional Bonus: Literature Screening

If you finish Section 3 with time to spare, extend it: accept a batch of article PDFs uploaded independently (not through the mailbox), decide whether each one describes a real, identifiable patient case worth reporting, split out multiple cases within one article if present, and produce a summary + relevance reason for each — reusing your Section 3 UI and logic where it makes sense. Not required, but scored as bonus credit.

5. Suggested Tech Stack

This mirrors what Clinevo actually uses in production. Deviating is fine if you have a good reason (e.g., a Python library with no Java equivalent) — just explain the choice.

Layer	Technology	Purpose
Frontend	Angular	The reviewer screen — queue, document viewer, editable extracted fields.
Backend API	Spring Boot (Java)	REST API, orchestration, security, saving data.
AI services	Python	Talks to the LLM/OCR models, does the extraction work, exposed to the Java layer over REST/gRPC.
Database	Oracle (PL/SQL)	Stores messages, extracted records, and the audit log.
AI model	Your choice	Any LLM/vision model, cloud or self-hosted — just document what you picked and why (cost, speed, accuracy trade-offs).

Overall flow: Angular → Spring Boot → Python AI service → Oracle DB. Use a simple queue (even an in-process one) rather than pure synchronous calls, since AI/OCR processing takes a few seconds to a minute per document. Include a basic architecture diagram in your write-up — hand-drawn is fine.

6. Test Data You'll Need to Create

All data must be made up / synthetic — never use real patient information.

1. At least 10 sample emails with varying levels of detail about a reaction.
2. At least 5 normal digital PDF attachments (e.g., a filled-in report form).
3. At least 2 scanned/handwritten-style PDFs (you can photograph a hand-filled mock form).
4. At least 5 made-up "article" PDFs describing a fictional patient case.
5. At least 2 PDFs in a non-English language with case-relevant content.
6. At least 2 quality-complaint-only and 2 info-request-only examples.
7. At least 1 clearly irrelevant example (e.g., a marketing email).

7. What to Submit

#	Deliverable	Notes
1	Working prototype	A live, runnable app (frontend + backend + DB) — not a slide deck or mockup.
2	Source code	Git repo or zip, with clear setup instructions.
3	README	How to run it locally, including any required environment variables (use placeholders, not real API keys).
4	Short write-up (2–5 pages)	Architecture diagram, tech choices, your prompting approach, known limitations, what you'd change for production.
5	Sample outputs	Extracted JSON per test document, plus screenshots or a short screen recording of the review screen.
6	Bonus (optional)	Literature screening extension from Section 4, if attempted.

8. How This Will Be Scored

Area	Weight	What we're looking for
Core functionality	30%	Email/PDF intake, all 4 PDF types, tables/images, sorting, and fact extraction working.
AI/LLM quality	25%	Good prompt design, structured outputs, sensible confidence scoring, saying "unknown" instead of guessing.
Code & architecture	20%	Clean separation across the stack, sensible API design, readable code.
Getting the domain right	10%	Correctly telling the 4 categories apart using the rules in Section 2.
Traceability & data handling	10%	Every fact links to its source; good audit logging; no real data used.
Documentation	5%	Clear README and write-up; can explain your trade-offs.
Bonus extension	+30%	Working literature screening from Section 4.

A working prototype with rough edges will always score higher than a polished document with nothing running — build first, polish second.